39 · 技术债与后续任务总清单

> 汇总 docs/27、33、35、37 中散落的全部遗留项，逐项标注可行性与处置结果 状态： 已关闭 ⚠ 部分关闭/结论修正 客观受阻（需外部条件） 记录为已知行为 执行结果见 40_技术债清零交付报告 与 41_上线就绪报告（含逐项证据、17 个缺陷、测试与运维变更） 2026-10-24 补充：诊断覆盖率 6/1200 → 1200/1200；docs/36 剩余 8 个模块缺口全部补齐； 安全加固（Secret + 强制鉴权 + 口令轮换）；NFR 三项实测达标。

一、总清单

#	来源	遗留项	类型	可行性	处置
1	37 §7	告警靠人工点「同步」，应改 CronJob	运维自动化	高	
2	38 §4	日常巡检靠手工敲 5 条命令	运维自动化	高	
3	37 §7	`rul_p50_days` 有字段无值，模型侧未产出	数据/模型	高	
4	37 §7 / 35 §6	无权限分级，所有视图同一账号	功能	中	
5	37 §7	点检巡检模块缺失	功能	中	
6	37 §7	保养计划模块缺失	功能	中	
7	35 §6	无暗色管理台	前端	高	
8	37 §7	组织树设备只到车间级，产线层为 0	数据	中	补挂产线（12×100）+ 修正 DFS 排序，见 40 §3
9	33 §9	python-diagnosis 单副本风险	容量	—	已自然恢复 2/2
10	33 §9	Egress 策略白名单需人工维护	运维	中	改为自动校验
11	33 §9	kube-proxy 规则在 iptables 1.4.21 显示不全	基础设施	—	
12	35 §6	无图表库，手写 SVG 扩展成本高	前端	—	刻意选择
13	35 §6	测试数据稀疏（仅 2 台有健康度）	数据	低	

#	来源	遗留项	类型	可行性	处置
14	33 §9 / 27	worker1 CPU 配额打满（96%）	容量	低	⚠ 结论修正：96% 是 requests，实际用量 8 – 16%；已下调本项目 requests，见 40 §5.4
15	27	K8s 版本升级（1.28 → 更高）	基础设施	低	容量不足
16	27	TiDB Operator 迁移	基础设施	—	已放弃（容量）

二、已关闭项的实施说明

2.1 告警自动同步 CronJob（#1）

问题：`/v1/alerts/sync` 幂等且安全，但要靠人在页面上点按钮。

方案：CronJob 每 10 分钟调用一次，用 `curl` 打内部 Service。

```
schedule: "*/10 * * * *"
command: curl -fsS -X POST http://python-diagnosis:8000/v1/alerts/sync \
           -H "X-Tenant-Id: site-a" -d '{"threshold_hi":70}'
```

注意：`alerts/sync` 不带 Basic Auth（它是内部 API，只校验租户头）， CronJob 在集群内直连 Service，不经 Ingress。

2.2 自动巡检脚本（#2）

问题：docs/38 §4.1 的巡检清单是 5 条手工 curl，容易漏。

方案：`impl/tools/health_check.py`，一次跑完 8 项检查，输出通过/失败与退出码， 可直接接监控告警。

2.3 RUL 估算（#3）

问题：`read_asset_health.rul_p50_days` 字段存在但全为 NULL。

方案：从 `read_asset_health_history` 算劣化斜率，线性外推到 HI=40（临界线）：

```
rul_days = (hi_now - 40) / max(0.01, degradation_per_day)
```

前提与边界：

- 历史不足 3 个点时不估算（返回 NULL，不臆造）
- HI 已低于 40 时 RUL = 0
- 斜率 ≤ 0（无劣化）时 RUL = NULL（视为"无劣化趋势"）

2.4 权限与角色（#4）

方案：轻量 RBAC，不引入完整 IdP。

- `app_user` 表：用户名、角色、口令哈希（PBKDF2）
- 登录接口签发 token（HMAC 签名，含角色与过期时间）

- 写操作按角色校验：`operator` 只能 ack，`supervisor` 可 escalate，

`manager` 可 close

- 前端顶部显示当前角色，无权限的按钮置灰

边界：不做细粒度资源级权限（如"只能管自己车间的设备"），那是 ABAC 范畴，超出本期。

2.5 点检巡检（#5）

表设计：

```
inspection_task(tenant_id, task_id, asset_id, plan_id, due_at, status,
                assignee, priority, created_at)
inspection_record(tenant_id, record_id, task_id, asset_id, result,
                  findings, measured, inspector, created_at)
```

流程：按关键度自动生成点检任务（A 类 7 天一次、B 类 15 天、C 类 30 天）→
值班员执行并记录结果（正常/异常/需停机）→ 异常项自动生成告警。

2.6 保养计划（#6）

表设计：

```
maintenance_plan(tenant_id, plan_id, asset_id, cycle_days, last_done_at,
                 next_due_at, items, owner_role, status)
```

流程：按周期计算下次到期，到期前 7 天标黄、到期未做标红；可从方案卡"计划窗口"一键落为保养计划。

2.7 暗色管理台（#7）

`console.html` 加 `prefers-color-scheme: dark` 分支 + 手动切换按钮，色值与 `screen.html` 的深色体系共用。

2.8 组织树产线级归属（#8）

问题：`asset.node_id` 只到车间（`site-a-W1`），产线层设备数为 0。

方案：不造假数据，改为按节点名前缀推断——设备位号里含产线标识时归到产线（当前数据无此标识），否则在页面明确标注「该层未直接挂设备」，并提供「按车间下钻」的交互。

实施：后端增加 `asset_by_node` 查询，前端点击车间节点展开其设备列表。

2.9 Egress 白名单自动校验（#10）

方案：`health_check.py` 增加一项：检查 `allow-egress-backends` 是否含 `podSelector: {}`，缺失则报警——避免再次出现"看板正常、服务间超时"。

三、客观受阻项说明

3.1 worker1 CPU 配额 96%（#14）

```
master   requests 15380m / 96%
worker1  requests 22320m / 34%
```

master 已接近打满，滚动更新会 Pending。根本解法是加节点或扩 CPU，当前物理条件不具备。可做的缓解：

- 降低非关键服务 requests（已做：python-diagnosis 降至 500m）
- 巡检脚本增加"CPU 配额余量"检查，低于 10% 时告警

3.2 K8s 升级（#15）

1.28 → 更高版本需要逐节点 drain，而单节点承载全部业务 Pod， drain 即全站中断。需先加节点，属容量问题。

3.3 TiDB Operator 迁移（#16）

已在 docs/27 记录：Operator 要求每副本 ≥ 2C4G，当前配额不足， 且托管模式会接管现有 StatefulSet，风险高于收益。已放弃，不再推进。

四、记录为已知行为（非缺陷）

4.1 kube-proxy 规则显示不全（#11）

`iptables-save` 在 iptables 1.4.21 上对 DNAT 规则显示为 `--to-destination --to-destination 0.0.0.0:0`，但实际转发正常。 排查网络问题时不要据此判断规则损坏，应以实际连通性测试为准。（docs/33 §2.1 有两次误判的完整记录。）

4.2 手写 SVG 而非图表库（#12）

内网无外网，CDN 不可达；内联 ECharts 会使单文件膨胀到 1MB+ 且受 ConfigMap 大小限制。当前 8 类图表手写实现，单文件 85KB。 若后续图表超过 15 个，再考虑内联轻量库（如 uPlot，约 40KB）。

4.3 测试数据稀疏（#13）

仅 2 台设备有健康度快照，导致大屏矩阵与趋势信息量受限。
属造数策略问题，非设计缺陷。可通过批量诊断（`/warmup`）扩充。